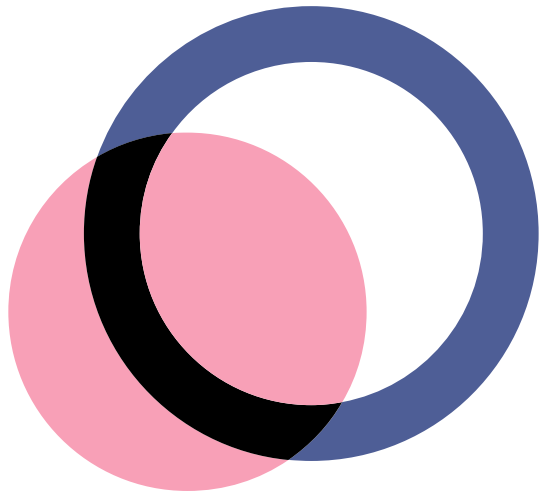


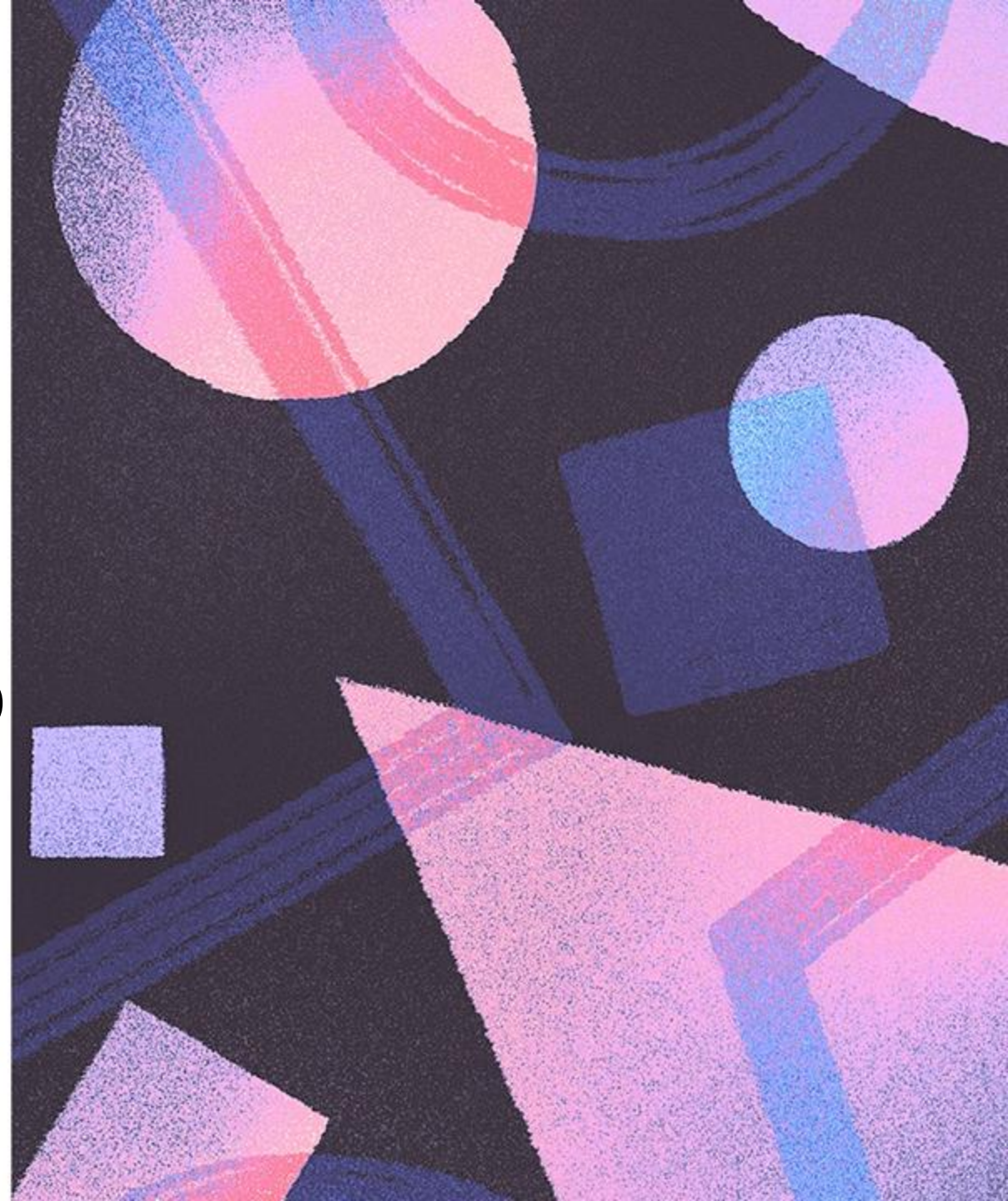


Design Patterns Lab

Lecture 5



Singleton Pattern



What is Singleton?

- Not just “one object exists”
- It means one shared object across the system
- The key idea: shared resource
- Used when all parts must see the same data

Why Do We Need It?

- Multiple copies → waste memory
- Repeated loading → slow system
- Different copies → inconsistent behavior
- Singleton ensures **one source of truth**

Real Scenario

- Your application has multiple features that can be turned ON/OFF
- Example: new UI, discount system, experimental search
- Different parts of the system need to check these decisions
- The system must behave differently based on these values

Problem Before Singleton

- Each service creates its own cache
- Example:
 - `new FeatureFlagCache()`
- Every service loads flags separately
- Creation + usage mixed in same class

Issues Without Singleton

- Same data loaded many times
- Memory duplication (many copies)
- Different services see different values
- Leads to real production bugs

What Singleton Solves

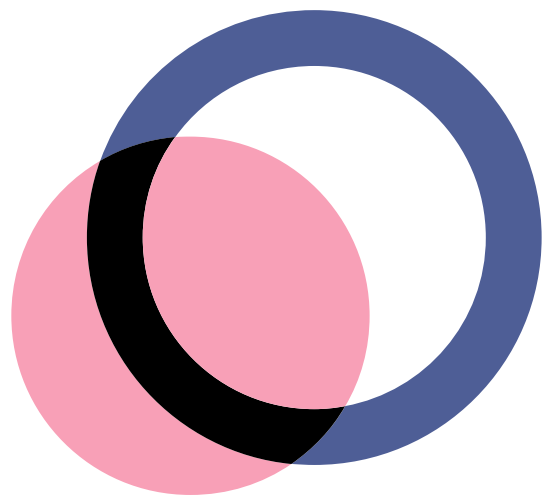
- Expensive object creation
- Shared system-wide data
- Consistent state across services
- Examples:
 - config, cache, logger

Problem with Singleton

- Creates tight coupling
- Class decides its dependency
- Hard to replace in testing
- Cannot easily use fake/mock objects

Activity: Logging System

- You are building a system with: OrderService, PaymentService, UserService
- All need to write logs to the same file
- Problem:
 - Each service creates its own logger
 - Logs become inconsistent and duplicated



**THANK
YOU**

